

Task 1:

```
class BankAccount:
    def __init__(self, accountno, name, balance):
        self.__accountno = accountno
        self.__name = name
        self.__balance = balance
    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
        else:
            print("Invalid deposit amount.")

    def withdraw(self, amount):
        if 0 < amount <= self.__balance:
            self.__balance -= amount
        else:
            print("Invalid or insufficient funds for withdrawal.")

    def set_accountno(self, accountno):
        self.__accountno = accountno

    def get_accountno(self):
        return self.__accountno

    def set_name(self, name):
        self.__name = name

    def get_name(self):
        return self.__name

    def get_balance(self):
        return self.__balance

    def set_balance(self, balance):
        if balance >= 0:
            self.__balance = balance
        else:
            print("Balance cannot be negative.")
```

```
a = BankAccount(1234,"surya",89000)
print(f"Account No:{a.get_accountno()}")
print(f"Account Name:{a.get_name()}")
print(f"Account Balance:{a.get_balance()}")
a.deposit(10000)
print(f"Account Balance After Deposit:{a.get_balance()}")
a.withdraw(10000)
print(f"Account Balance After Withdraw:{a.get_balance()}")
```

```
py
● Account No:1234
  Account Name:surya
  Account Balance:89000
  Account Balance After Deposit:99000
  Account Balance After Withdraw:89000
○ PS C:\Users\surya.p\Desktop\PythonTask> █
```

Task 2:

```
class MathOps:
    @staticmethod
    def add(x, y):
        return x + y

print(f"Total:{MathOps.add(5, 3)}")
```

```
PS C:\Use
2.py
Total:8
```

Task 3:

- `__repr__(self)`: Returns an official string representation of the object.
- `__len__(self)`: Returns the length of the object (used with `len()`).
- `__getitem__(self, key)`: Allows bracket indexing (e.g., `obj[key]`).
- `__setitem__(self, key, value)`: Allows assignment via brackets.
- `__call__(self, ...)`: Makes an object callable as a function.
- `__eq__(self, other)`: Defines behavior for the equality operator `==`.
- `__add__(self, other)`: Behavior for the `+` operator.

Task 4:

Supervised Learning Models

- These algorithms learn from labeled training data—each input has a known output (target).
- Linear Regression: Predicts a continuous outcome.
- Logistic Regression: Classification, predicts binary classes.
- Decision Trees: Splits data based on features for prediction/classification.
- Random Forest: Ensemble of decision trees for better accuracy.
- Support Vector Machines (SVM): Finds hyperplanes to classify data.
- K-Nearest Neighbors (KNN): Classifies based on similarity to nearby points.

Unsupervised Learning Models

- These work with unlabeled data to find patterns or structure.
- K-Means Clustering: Partitions data into k distinct groups (clusters).
- Hierarchical Clustering: Builds clusters incrementally using a tree structure.
- Principal Component Analysis (PCA): Reduces dimensionality, highlighting variance in data.
- DBSCAN: Density-based clustering, forming clusters based on data density.
- Autoencoders: Neural networks for dimensionality reduction and feature learning.

Task 5:

```
class Stack:
    def __init__(self):
        self.stack = []

    def push(self, element):
        self.stack.append(element)

    def pop(self):
        if self.isEmpty():
            return "Stack is empty"
        return self.stack.pop()

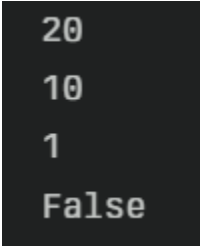
    def peek(self):
        if self.isEmpty():
            return "Stack is empty"
```

```
        return self.stack[-1]

    def isEmpty(self):
        return len(self.stack) == 0

    def size(self):
        return len(self.stack)

s = Stack()
s.push(10)
s.push(20)
print(s.pop())
print(s.peek())
print(s.size())
print(s.isEmpty())
```



```
20
10
1
False
```